

UNIVERSITÉ DE TOULOUSE
Master BI/BEE – Mention Bioinformatique et Génomique
Environnementale

Analyse et Manipulation de la Gene Ontology

*Modélisation de graphe
de Rattus norvegicus*

Auteur :
Florent LE QUELLEC

Cadre du projet :
UE Algorithmique des
Graphes

Année universitaire 2024–2025



Licence **Creative Commons BY–NC 4.0**

Table des matières

1	Contexte	1
2	Analyse	1
3	Conception	1
3.1	Représentation des données	1
3.2	Fonctionnalités développées	2
3.3	Module spécifique Gene Ontology	3
3.4	Algorithmes	3
3.4.1	Algorithme : GOTerms	3
3.4.2	Algorithme : GeneProducts	4
3.4.3	Algorithme : Max Depth (Optimisé)	5
4	Résultats	7
5	Discussion	7
6	Bilan et Perspectives	7
7	Section spéciale : Méthodologie de travail	8

1 Contexte

Ce projet s'inscrit dans le cadre de l'UE *Graphes*, dont l'objet est de fournir des méthodes d'analyse pour des volumes de données conséquents. L'objectif principal est le développement de bibliothèques Python permettant la modélisation et l'exploration de la *Gene Ontology* (GO). Ce projet bioinformatique vise à standardiser et structurer la description des gènes et des produits géniques à travers les différents organismes.

Pour cette étude, nous avons sélectionné *Rattus norvegicus*, communément appelé rat brun. Organisme modèle incontournable dans la communauté scientifique, il est, à l'instar de la souris, prédominant dans les expérimentations *in vivo*, notamment en pharmacologie et en recherche biomédicale. En conséquence, son génome est extrêmement bien documenté ; il fut d'ailleurs l'un des premiers mammifères à être entièrement séquencé en 2004.

2 Analyse

Notre analyse repose sur l'exploitation conjointe de deux sources de données :

- Le fichier **OBO** : il contient les informations structurelles du graphe, à savoir les termes *Gene Ontology* et leurs relations (arcs).
- Le fichier **GOA** : il contient les annotations spécifiques à *Rattus norvegicus*, liant les produits de gènes aux termes du fichier OBO.

Il nous faut donc d'une part pouvoir analyser et modéliser des graphes de manière générique, et d'autre part disposer d'outils pour traiter et analyser la syntaxe des fichiers spécifiques à la GO. Pour répondre à ces besoins, nous allons développer deux modules distincts : la bibliothèque *gm.py* (GraphMaster) pour la gestion mathématique des graphes, et le module *geneontology.py* pour l'exploitation des données biologiques.

3 Conception

3.1 Représentation des données

Au vu de la nature hiérarchique et interconnectée de nos données, le choix le plus pertinent est une représentation sous forme de **graphe orienté**. Dans ce modèle :

- Les **nœuds** représentent soit des termes de l'ontologie (*GOTerm*), soit des produits géniques (*GeneProduct*).

- 30 — Les **arcs** représentent les relations sémantiques (orientées de l'enfant vers le parent pour *is_a* et *part_of*) ou les annotations (du gène vers le terme).

Pour mettre en œuvre ce projet, nous avons conçu deux librairies Python distinctes : la première dédiée à la manipulation générique de graphes, et la seconde spécialisée dans le traitement des données de la *Gene Ontology* (GO).

3.2 Fonctionnalités développées

L'ensemble du projet a été réalisé avec Python 3 et les librairies standards telles que `re`, `sys` et `os`. Cependant, deux librairies tierces sont nécessaires : `pandas` et `polars`. Elles sont utilisées au sein de la fonction `read_delim` pour optimiser les performances de lecture.

- 40 Dans notre première librairie, nommée *gm.py* (GraphMaster), nous avons implémenté une classe *graph* qui encapsule les informations suivantes :

- Le dictionnaire des nœuds et leurs attributs.
- Le dictionnaire des arcs (liste d'adjacence) et leurs poids éventuels.
- Le caractère dirigé (orienté) ou non du graphe.

Les méthodes suivantes ont été implémentées pour manipuler cette structure :

- *add_node* / *add_edge* : Permettent l'ajout dynamique de nœuds et d'arcs.
- *node_exists* / *edge_exists* : Vérifient la présence d'un élément dans le graphe.
- *nodes* : Renvoie la liste triée de tous les nœuds.
- *nb_nodes* / *nb_edges* : Renvoient respectivement le nombre total de sommets et d'arêtes.
- 50 — *neighbors* : Renvoie la liste des voisins (successeurs) d'un nœud donné.
- *predecessors* : Renvoie la liste des prédécesseurs d'un nœud (utile pour remonter les relations parents-enfants).
- *read_delim* : Fonction utilitaire pour charger un graphe depuis un fichier texte délimité (CSV/TXT).
- *BFS* (Breadth-First Search) : Parcours en largeur, permettant notamment de calculer les plus courts chemins depuis un nœud source.
- *DFS* (Depth-First Search) : Parcours en profondeur, utilisé pour l'exploration exhaustive des branches.
- 60 — *connected_components* : Identifie les sous-graphes disjoints (pour les graphes non orientés).
- *edges_filter* : Crée un sous-graphe en filtrant les arcs selon un critère de poids.
- *clustering_coefficient* : Calcule le coefficient de clustering (transitivité locale) d'un nœud.

- *is_acyclic* : Vérifie si le graphe est un DAG (Directed Acyclic Graph).
- *topological_sort* : Propose un ordre linéaire des nœuds respectant la direction des arcs.

3.3 Module spécifique Gene Ontology

Notre seconde librairie, *geneontology.py*, utilise les fonctions de *gm.py* pour manipuler les données biologiques. Elle implémente les fonctions métiers suivantes :

- *load_OBO* : Analyse la syntaxe du fichier de structure de l'ontologie (.obo).
- *load_GOA* : Charge le fichier d'annotations (.goa) et lie les produits géniques aux termes GO correspondants.
- *GOTerms* : Récupère les termes GO associés à un produit génique donné (avec récursion optionnelle).
- *GeneProducts* : Récupère les produits géniques associés à un terme GO (avec récursion optionnelle).
- *max_depth* : Calcule la profondeur maximale des trois sous-ontologies (BP, MF, CC) en utilisant une optimisation par index inversé.

3.4 Algorithmes

3.4.1 Algorithme : GOTerms

Cet algorithme décrit la remontée vers les ancêtres (parents) pour trouver tous les termes associés à un gène.

Algorithme 1 : Récupération des termes GO d'un produit génique

Entrées Graphe $G = (V, E)$, Identifiant Gène g , Booléen *recursif*

Sortie : Ensemble des termes GO associés T

```
1  $T \leftarrow \text{VoisinsSortants}(G, g);$ 
2 if recursif est Vrai then
3    $File \leftarrow \text{Liste}(T);$ 
4    $Visites \leftarrow T;$ 
5   while  $File$  n'est pas vide do
6      $u \leftarrow \text{Defiler}(File);$ 
7      $Parents \leftarrow \text{VoisinsSortants}(G, u);$            // Relation is_a
8     foreach  $p \in Parents$  do
9       if  $p \notin Visites$  then
10        Ajouter  $p$  à  $T$ ;
11        Ajouter  $p$  à  $Visites$ ;
12        Enfiler  $p$  dans  $File$ ;
13 return  $T$ ;
```

3.4.2 Algorithme : GeneProducts

Ici, on décrit la descente vers les termes plus spécifiques (descendants) pour agglomérer tous les gènes concernés.

Algorithme 2 : Récupération des gènes associés à un terme GO

Entrées Graphe $G = (V, E)$, Identifiant Terme t_{racine} , Booléen *recursif*

Sortie : Ensemble des gènes P

```
1 TermesCibles  $\leftarrow \{t_{racine}\};$ 
  /* Étape 1 : Expansion des termes (Descente vers les
    enfants) */
2 if recursif est Vrai then
3   File  $\leftarrow \{t_{racine}\};$ 
4   Visites  $\leftarrow \{t_{racine}\};$ 
5   while File n'est pas vide do
6      $u \leftarrow \text{Depiler}(\text{File});$ 
7     Enfants  $\leftarrow \text{Predecesseurs}(G, u);$  // Sens inverse des arcs
8     foreach  $v \in \text{Enfants}$  do
9       if  $\text{Type}(v) == \text{"GOTerm"}$  et  $v \notin \text{Visites}$  then
10        Ajouter  $v$  à TermesCibles;
11        Ajouter  $v$  à Visites;
12        Empiler  $v$  dans File;
  /* Étape 2 : Collecte des gènes annotés */
13  $P \leftarrow \emptyset;$ 
14 foreach  $t \in \text{TermesCibles}$  do
15   Sources  $\leftarrow \text{Predecesseurs}(G, t);$ 
16   foreach  $s \in \text{Sources}$  do
17     if  $\text{Type}(s) == \text{"GeneProduct"}$  then
18       Ajouter  $s$  à  $P$ ;
19 return  $P;$ 
```

3.4.3 Algorithme : Max Depth (Optimisé)

90 Cet algorithme détaille la phase de pré-calcul (index inversé) et la récursion avec mémoïsation.

Algorithme 3 : Calcul de la Profondeur Maximale (Optimisé)

Entrées Graphe $G = (V, E)$, Liste des Racines $R = \{BP, MF, CC\}$

Sortie : Dictionnaire des profondeurs D

```
/* 1. Pré-calcul de l'index inversé (Optimisation
   complexité) */
```

1 *IndexEnfants* $\leftarrow$ DictionnaireVide();2 **foreach** $Arc(u, v) \in E$ **do**

```
3  | Ajouter  $u$  à  $IndexEnfants[v]$ ;           //  $u$  est enfant de  $v$ 
```

```
4 Memo ← DictionnaireVide();
```

```
/* 2. Définition de la fonction récursive interne */
```

5 Fonction Hauteur (u) :

6 | **if** $u \in Memo$ **then**

```

7      |      | return Memo[u];

```

8	$Candidats \leftarrow IndexEnfants[u];$
---	---

9	$EnfantsValides \leftarrow \emptyset;$
---	--

```

10  foreach  $c \in Candidates$  do

```

```

11      if  $Type(c) \neq "GeneProduct"$  then

```

12		Ajouter c à $EnfantsValides$;
----	--	----------------------------------

13	if <i>EnfantsValides</i> <i>est vide</i> then
----	---

14		$Memo[u] \leftarrow 0;$
-----------	--	-------------------------

15	return 0;
----	------------------

16	$MaxH \leftarrow 0;$
----	----------------------

```

17  foreach enfant ∈ EnfantsValides do

```

18	$h \leftarrow \text{Hauteur}(\text{enfant}) ;$
----	--

19	if $h > MaxH$ then
----	----------------------------------

20			$MaxH \leftarrow h;$
----	--	--	----------------------

21	$Memo[u] \leftarrow 1 + MaxH;$
----	--------------------------------

```

22   return  $1 + MaxH$ ;

```

```
/* 3. Exécution principale */
```

```
23 D ← DictionnaireVide();
```

24 **foreach** $racine \in R$ **do**

25 | Vider *Memo*;

26 $D[racine] \leftarrow \text{Hauteur}(racine) ;$

```

27 return  $D$ ;

```


4 Résultats

Les résultats obtenus avec les données de *Rattus norvegicus* sont les suivants :

- **43 328** termes GO chargés.
- **76 581** relations (*is_a*, *part_of*).
- **47 072** gènes (*GeneProducts*) ajoutés au graphe.

L'analyse topologique révèle les profondeurs maximales suivantes :

- Biological Process : 18
- Molecular Function : 12
- Cellular Component : 14

Ces résultats confirment la richesse de l'annotation du génome du rat brun, caractérisée par un nombre important de gènes et une grande profondeur dans l'arbre ontologique.

5 Discussion

En analysant les résultats, on remarque un ratio important entre le nombre de gènes et les termes GO. Cela implique une forte redondance : un même terme annote souvent de très nombreux gènes, formant ainsi des centres génétiques. Cette densité peut s'expliquer par une surreprésentation de certains gènes dans l'annotation.

En particulier, l'analyse des profondeurs montre que le domaine des *Biological Processes* (BP) est le plus profond. Compte tenu de l'utilisation du rat (*Rattus norvegicus*) comme organisme modèle en médecine, on peut supposer un biais d'étude vers les mécanismes pathologiques (cancer, maladies humaines). À l'inverse, la *Molecular Function*, moins sujette à ces contextes physiopathologiques complexes, présente logiquement la profondeur la plus faible des trois domaines.

6 Bilan et Perspectives

Les besoins initiaux du projet ont été satisfaits. Les modules développés permettent de charger les données de la *Gene Ontology*, d'obtenir les statistiques fondamentales (nombre de termes GO, produits géniques chargés) et d'effectuer des calculs topologiques comme la profondeur des sous-ontologies.

Le choix de Python, langage de haut niveau, a facilité l'implémentation des structures de données. Si cette accessibilité a permis un développement rapide, certains algorithmes se sont révélés coûteux en temps de calcul sur des volumes de données importants, nécessitant une optimisation.

Les bibliothèques produites restent perfectibles, notamment concernant la visualisation graphique des réseaux qui n'a pas été abordée ici, une interface graphique client pourrait très bien être développée. Néanmoins, ce travail constitue une base fonctionnelle pour mener des études d'annotation fonctionnelle, de phylogénie, d'analyse médicale ou encore de génomique comparative.

130 7 Section spéciale : Méthodologie de travail

Dans le cadre de ce projet, j'ai travaillé en autonomie pour l'analyse, la conception et l'implémentation des modules. J'ai également consulté les supports de cours, le projet de Camille-Astrid RODRIGUES (https://github.com/CamilleAstrid/Algorithmes_de_Graphes_et_Annotation_Fonctionnelle-Etude_Proteomique_Rattus_norvegicus) ainsi que la documentation de la *Gene Ontology* pour comprendre la structure des fichiers OBO.

Conformément aux consignes, j'ai eu recours à l'intelligence artificielle générative (Gemini) comme assistant de développement. Ce recours a été transparent et s'est limité à trois aspects spécifiques :

- 140 — **Débogage syntaxique** : Aide à la correction d'erreurs Python et à la gestion des exceptions lors du parsing des fichiers textes.
- **Optimisation algorithmique** : C'est sur ce point que l'aide a été la plus cruciale. Lors du développement de la fonction `max_depth`, la première version de l'algorithme demandait trop de ressources (temps de calcul excessif lié à une complexité quadratique). Gemini m'a aidé et a proposé une solution basée sur un pré-calcul des relations (index inversé).
- **Rédaction technique et orthographique** : Assistance pour la formalisation des algorithmes en pseudo-code (utilisés dans ce rapport), la vérification de la clarté des docstrings et la correction du rapport.

150 J'ai configuré l'assistant avec une consigne imposant des contraintes pédagogiques strictes :

- **Interdiction de fournir du code source** : L'IA avait pour consigne explicite de « *ne jamais fournir de code directement* » mais exclusivement du pseudocode ou

des descriptions algorithmiques, m'obligeant à traduire moi-même la logique en Python.

- **Rôle d'expert-conseil** : L'assistant a été paramétré pour agir en tant qu'« *expert en bioinformatique spécialisé dans la théorie des graphes* », avec pour but de guider la conception sans donner la solution finale.
- **Ancrage dans le cours** : Les explications devaient faire le lien avec les concepts théoriques du cours et les structures de données du TP (listes d'adjacence, dictionnaires), assurant une certaine cohérence avec l'enseignement.

160